Associations and Factories

Factory_bot has a powerful set of features for adding associations to factories. I'll talk about them because they are so powerful, and you might see code that uses them. Then I'll talk about why you should be careful about using them.

The simplest case is also a common one: the class being created has a belongs_to association with the same name as a factory. In that case, you just include that name in the factory (these code snippets aren't in the code sample, for reasons you'll learn by the end of this section):

```
FactoryBot.define do
  factory :task do
    title "Do Something"
    size 3
    project
  end
end
```

In this case, calling task = FactoryBot.create(:task) would also implicitly call task.project = FactoryBot.create(:project).

If you want to explicitly specify the project when calling the task factory, you can do so the same way you would for any other attribute—namely, via task = FactoryBot.create(:task, project: Project.new). If the association is specified in the factory definition but you don't want any value in the test, you need to explicitly set the association to nil, as in: task = FactoryBot.create(:task, project: nil).

If the association name doesn't match the factory name or if you want to specify default attributes of the associated object, you can explicitly use the association method in the factory definition:

```
FactoryBot.define do
  factory :task do
    title "Do Something"
    size 3
    project
    association :doer, factory: :user, name: "Task Doer"
  end
end
```

The syntax is association, followed by the name of the association in the ActiveRecord model and then a bunch of key/value pairs, with the factory key being used by factory_bot to determine which factory to use to create the associated object.

The way in which the subordinate object is built depends on the build strategy used for the parent object. The problem lies in the way ActiveRecord